

Experiment 4

Name : Arjun Dubey

Branch: BE-CSE

Semester: 6th

Subject Name: Full Stack-II

UID 23BCS13190

Section/Group: KRG-3B

Date of Performance:-06/02/2026

Subject Code: 23CSH-309

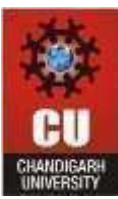
Aim: The aim of this implementation is To develop a secure navigation system using Context-based authentication, protected routing, and optimized React hooks to improve performance and UI using Material UI.

Objective:-

- To centrally handle authentication state across the entire application using Context API
- To configure client-side routing for seamless page navigation
- To safeguard protected routes by preventing access from unauthorized users
- To enable navigation between pages only after successful user authentication
- To automatically redirect users who are not logged in to the login page
- To enhance overall application security and user experience
- To ensure a clean, modular, and maintainable code architecture

Implementation/Code:

The **EcoTrack application** uses **Redux Toolkit** for centralized state management, ensuring predictable data flow and easier debugging. A global Redux store is configured using `configureStore` with feature-based slices like `logsSlice`, while asynchronous operations are handled using **Redux Async Thunks** to manage API request states. **React-Redux hooks** (`useDispatch`, `useSelector`) connect components to the store, enabling data fetching and state access. **Selectors** derive filtered views from the Redux state without mutation, and loading and error states are managed to improve performance and user experience.



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

- /store/logSlice.js :-

```
EcoTracker > src > store > JS logsSlice.js > ...
1  import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
2
3  export const fetchLogs = createAsyncThunk(
4    'logs/fetchLogs',
5    async () => {
6      await new Promise((resolve) => setTimeout(resolve, 1000));
7
8      return [
9        { id: 1, activity: 'Car Travel', carbon: 4 },
10       { id: 2, activity: 'Electricity Usage', carbon: 7 },
11       { id: 3, activity: 'Cycling', carbon: 0 },
12     ];
13   }
14 )
15
16 const logsSlice = createSlice({
17   name: 'logs',
18   initialState: {
19     data: [],
20     status: 'idle',
21     error: null,
22   },
23   reducers: {},
24   extraReducers: (builder) => {
25     builder.addCase(fetchLogs.pending, (state) => {
26       state.status = 'loading';
27     })
28     .addCase(fetchLogs.fulfilled, (state, action) => {
29       state.status = 'succeeded';
30       state.data = action.payload;
31     })
32     .addCase(fetchLogs.rejected, (state, action) => {
33       state.status = 'failed';
34       state.error = action.error.message;
35     })
36   },
37
38   // Optional: Add default reducers if any
39   // reducers: {
40   //   // Example reducer
41   // },
42
43   export default logsSlice.reducer;
```

- store.js :-

```
EcoTracker > src > store > JS store.js > ...
1  import { configureStore } from '@reduxjs/toolkit';
2  import logsReducer from './logsSlice';
3  const store = configureStore({
4    reducer: {
5      logs: logsReducer,
6    },
7  });
8  export default store;
```

- **LogSlice.jsx :-**

```
EcoTracker > src > JS logSlice.js > [⌕] default
1  import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
2  export const fetchLogs = createAsyncThunk("logs/fetchLogs", async () => {
3    return new Promise((resolve) =>
4      setTimeout(() => resolve([
5        { id: 1, activity: "Commute", carbon: 2.5 },
6        { id: 2, activity: "Lunch", carbon: 0.5 },
7      ]), 1000)
8    );
9  });
10
11 const logSlice = createSlice({
12   name: "logs",
13   initialState: {
14     data: [],
15     status: "idle", // 'idle' | 'loading' | 'succeeded' | 'failed'
16     error: null,
17   },
18   reducers: {},
19   extraReducers: (builder) => {
20     builder
21       .addCase(fetchLogs.pending, (state) => {
22         state.status = "loading";
23       })
24       .addCase(fetchLogs.fulfilled, (state, action) => {
25         state.status = "succeeded";
26         state.data = action.payload;
27       })
28       .addCase(fetchLogs.rejected, (state, action) => {
29         state.status = "failed";
30         state.error = action.error.message;
31       });
32   },
33 });
34
35 export default logSlice.reducer;
```

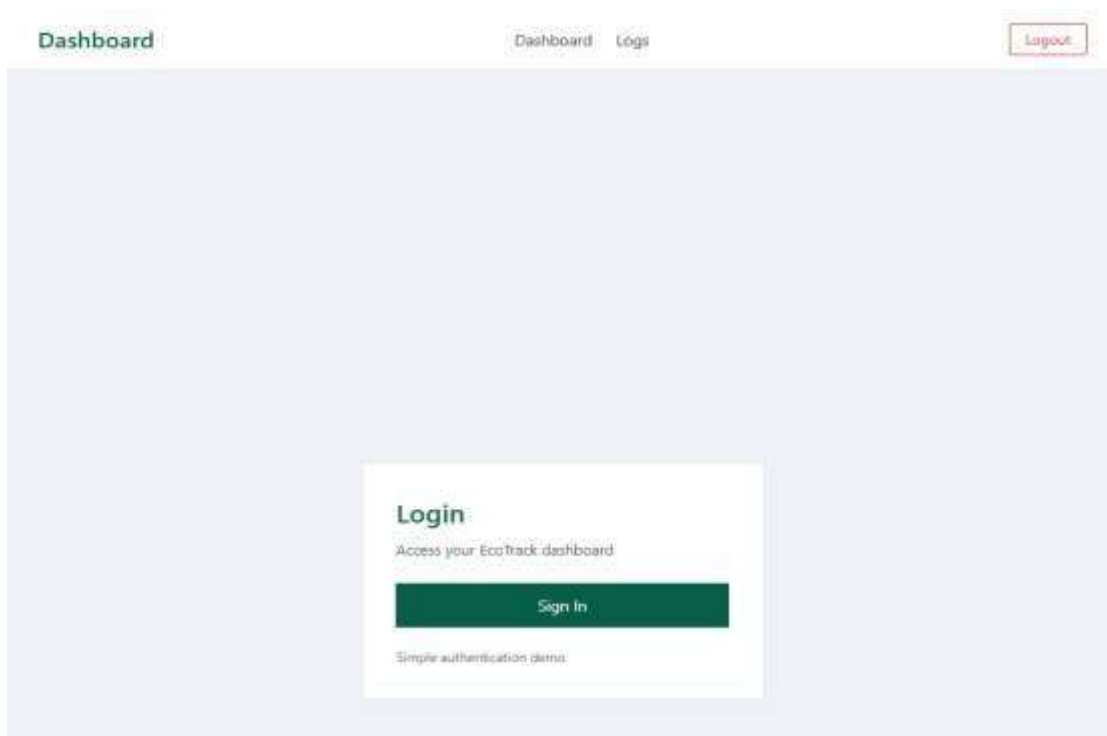
- App.jsx:-

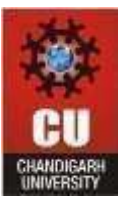
```
EcoTracker > src > App.jsx > App
1
2 import './App.css'
3 import Header from './components/Header'
4 import Dashboard from './pages/Dashboard.jsx'
5 import { BrowserRouter, Route, Routes } from 'react-router-dom'
6 import ProtectedRoute from './routes/ProtectedRoute'
7 import DashboardSummary from './pages/DashboardSummary'
8 import DashboardAnalytics from './pages/DashboardAnalytics'
9 import Login from './pages/Login'
10 import Logs from './pages/Logs'
11
12 function App() {
13
14   return (
15     <main>
16     <>
17     <BrowserRouter>
18     <Header title="EcoTrack"/>
19
20     <Routes>
21
22       <Route path="/login" element={(<Login/>) } />
23
24       <Route path="/" element={
25         <ProtectedRoute>
26           <Dashboard/>
27         </ProtectedRoute>
28       }>
29
30       <Route index element={(<DashboardSummary/>) } />
31       <Route path="summary" element={(<DashboardSummary/>) } />
32       <Route path="analytics" element={(<DashboardAnalytics/>) } />
33
34     </Route>
35     <Route path="logs" element={(<Logs/>) } />
36
37   </Routes>
38
39 </BrowserRouter>
40 </>
41 </main>
42
43 )
44
45 }
46
47 export default App
48
49
```

- ProtectRoute.jsx

```
1  import {Navigate} from "react-router-dom";
2  import {useAuth} from "../context/AuthContext";
3
4  const ProtectedRoute = ({children}) => {
5      const {isAuthenticated} = useAuth();
6
7      if(!isAuthenticated){
8          return <Navigate to="/login" replace />;
9      }
10     return children;
11 }
12 export default ProtectedRoute;
```

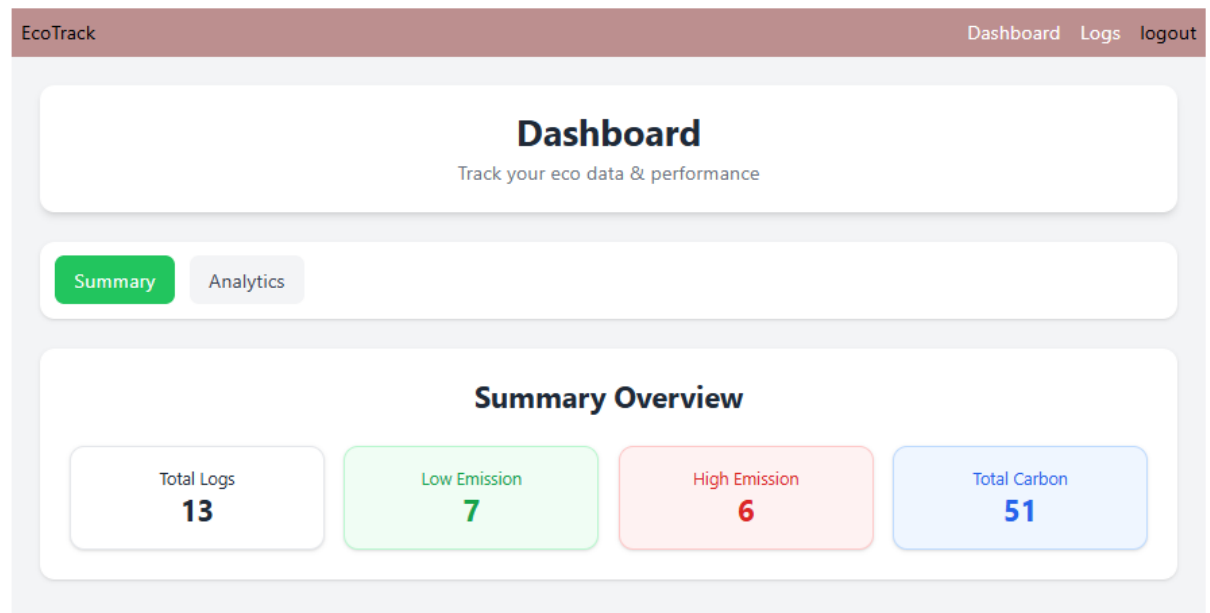
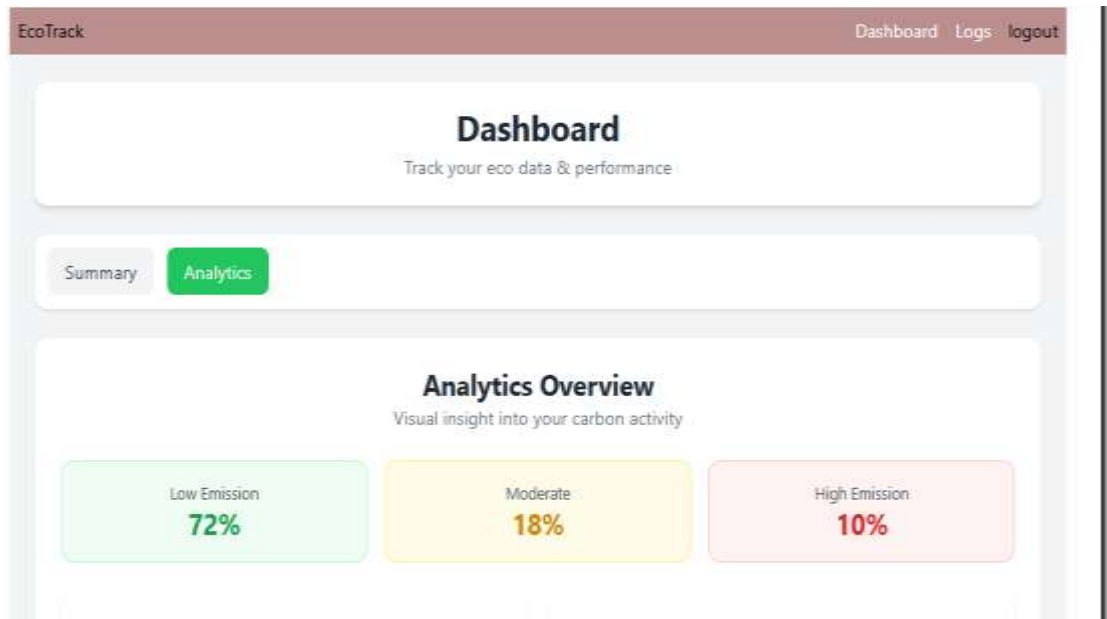
4. Output

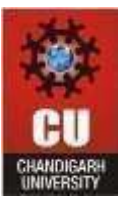




DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.





DEPARTMENT OF

COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

5. Learning Outcome :-

- Learned performance optimization using React.memo
- Implemented secure routing with context API
- Built modern UI using Material UI
- Improved scalability and maintainability